

Unit 04: Operations

CONTENTS

Objectives

Introduction

4.1 Operators

4.2 Unary operator

4.3 Binary operator

4.4 Arithmetic Operators

4.5 Relational Operators

4.6 Logical Operators

4.7 Assignment Operators

4.8 Increment and Decrement Operators

4.9 Conditional Operators

4.10 Bitwise Operators

4.11 Special Operators

4.12 Operator Precedence and Associativity

Summary

Keywords

Self-Assessment

Answer for Self Assessment

Review Questions

Further Readings

Objectives

After studying this unit, you will be able to:

- Various Data types
- Explain arithmetic operators
- Describe conditional, logical and relational operators
- Describe arithmetic expression

Introduction

The term "data types" refers to a comprehensive framework for defining variables and functions of various types. The type of a variable decides how much storage space it takes up and how the stored bit pattern is interpreted. An operator is a symbol that instructs the machine to perform mathematical or logical operations on data held in variables. A value's data form identifies a set of properties that it shares.

An expression is a set of constants, variables, and operators that follow the grammatical rules of the language C and evaluate to a valid value. Activity refers to the impact that operators have on their operands.

Programming In C

Valid or well-formed expression refers to an expression that follows the grammar rules of the C programming language. An expression that is true or well evaluates to a single value of a valid C data form. C expression can be of the following types:

1. Numerical expressions always evaluates to a numeric value on which arithmetic operations can be performed. They can be further divided into the following two categories:

(a) Integer expression: those evaluating to integer value

(b) Real expression: those evaluating to a real (floating point) value

Thus, $3 + 5$ is an integral expression and $3.8 - 6.97$ is a real expression.

2. Logical or conditional expressions always result into either of the two values – true or false.

Thus $3 > 5$ and $x \leq 7$ are conditional expressions.

4.1 Operators

An operator is a symbol that instructs the machine to perform mathematical or logical operations on data held in variables. Data items (variables and constants) can be manipulated arithmetically in a C program using operators.

C operators can be classified into a number of categories.

- Unary operator
- Binary operator
- Arithmetic operators
- Relational operators
- Logical operators
- Conditional operators
- Assignment operators
- Bitwise operators
- Increment and decrement operators
- Special operators

4.2 Unary operator

Unary operators are operators that act upon a single operand to produce a new value.

There are following types unary operators in C

Unary Operator	Meaning
increment(++)	Increment value of variable by 1
decrement(--)	Decrement value of variable by 1
sizeof()	Returns the size of its operand, in bytes.
Addressof operator(&)	Give an address of a variable.
NOT(!)	Used to reverse the logical state of its operand.
Unary minus (-)	Changes sign of its argument.

4.3 Binary operator

Binary operators are those that operate on two operands. For example, $a + b$ – the addition operator (+) surrounded by two operands – is a popular binary expression. Arithmetic, relational, logical, and assignment operators are all subsets of binary operators.

The binary operators are categorized as follow:

- Multiplicative operators: multiplication (*), remainder (%), and division (/)
- Additive operators: addition (+) and subtraction (-)
- Shift operators: left shift (<<) and right shift (>>)
- Relational operators: less than (<), less than or equal to (<=), greater than (>), and greater than or equal to (>=)
- Equality operators: equality (==) and inequality (!=)
- Bitwise operators: AND (&), OR (|), and XOR (^)
- Logical operators: AND (&&) and OR (||)

4.4 Arithmetic Operators

Arithmetic operators are used to perform arithmetic operations include addition, subtraction, multiplication and division on one or more operands.

It is used to performed operations on numeric data types like int, float and double Arithmetic operators work on numeric type of operands. C provides all the basic arithmetic operators. There are five arithmetic operators in C.

Operator	Purpose
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Remainder after integer division

The division operator (/) requires the second operand to be non-zero, though the operands need not be integers. When an integer is divided by another integer, the result is also an integer. In such instances the division is termed as integer division. Consider the following:

```
int x;
```

```
x = 10;
```

What do you expect the value of $x/4$ to be? If you guessed 2.5, you are wrong.

The result is of course 2.5 however, since it is integer division (division operation in which both the operands are integers), the result 2.5 will be truncated to 2 to make the result an integer. In case you wish to get the correct value you make this division a float type as $x/4.0$.

The % operator is known as modulus operator. It produces the remainder after the division of two operands. The second operand must be non-zero.

Rest all the other operators work in their normal arithmetic way. Normal BODMAS rules are also applicable to these arithmetic operators.

4.5 Relational Operators

The relational operator compares two operands to see if they are equivalent, unequal, or if one is greater or inferior to the other.

The consequence is always a numeric value equal to true or false, regardless of whether the operands are variables, constants, or expressions. As previously stated, a non-zero result indicates fact, while a zero result indicates false. Six relational operators are available in the C programming language.

==	equal to
!=	not equal to
<	less than
<=	less than or equal to
>	greater than
>=	greater than or equal to

A simple relation contains only one relational expression and takes the following form: $\langle \text{ae-1} \rangle \langle \text{relational operator} \rangle \langle \text{ae-2} \rangle$. $\langle \text{ae-1} \rangle$ and $\langle \text{ae-2} \rangle$ are arithmetic expressions, which may be constants, variables or combination of these. The value of the relational operator is either 1 or 0. If the relation is true, result is 1 otherwise it is 0.

Example:

Expressions	Result
$6.3 \leq 15$	True
$2.5 < -2$	False
$-10 \geq 0$	False
$10 < 8+3$	True

4.6 Logical Operators

Using logical operators, several relational expressions may be combined to create a single compound relational expression. To combine two or more relational statements, logical operators are used. Three logical operators are available in C. A compound relation behaves in the same way as a single relation generating a real or false value

Operator	Meaning	Result
&&	Logical AND	True if both the operands are true
	Logical or	True if both the operands are true
!	Logical not	True if the operand is false and vice versa



- (age > 50 && weight < 80) will evaluate to true if age is more than 50 and also weight is less than 80. Otherwise, it will evaluate to false.
- (a < 0 || ch == 'a') will evaluate to true if a is less than 0 while ch is equal to 'a', false otherwise.
- !(a < 0) will evaluate to true if a is greater than or equal to 0, false otherwise.

4.7 Assignment Operators

Assignment operators are used to store the result of an expression to a variable. The most commonly used assignment operator is (=). Be careful not to mistake assignment operator (=) for mathematical equality operator which is indicated by the same symbol.

An expression with assignment operator is of the following form:

`<identifier> = <expression>;`

When this statement is executed, `<expression>` is evaluated and the value is stored in the `<identifier>`.

Let us consider the following usage of assignment operator in C language.

```
int i;
i = 5;
i = i + 10;
```

The value now stored in the variable "i" will be 15. In this program, the current value stored in variable i is 5. Thus, while executing `i = i+10`, the right-hand side will be evaluated to give a value 15. This value will then be assigned to the left-hand side. As a result, the current value of i after execution of this statement will become 15.

C language provides a short cut to write arithmetic assignment expressions which takes the following form:

`<Variable> op = <expression>;`

This statement is identical to:

`<Variable> = <Variable> op <expression>;`

Thus, `i=i+3` can be written as `i+=3`

`Sum=sum-2` can be written as `sum-=2`

`i=i*5` can be written as `i*=5`

The advantages of using this form of assignment operators are:

1. The statement is more efficient and easier to read.
2. What appears on the L.H.S need not to be repeated and therefore it becomes easier to write for long variable names. Consider the following C code that illustrates this point.

```
int averylongvariablename;
averylongvariablename = 2;
while (averylongvariablename < 20)
{
    averylongvariablename*= averylongvariablename;
}
```

4.8 Increment and Decrement Operators

C has two very useful operators ++ and -- called increment and decrement operators respectively.

These are generally not found in other languages. These operators are unary operators as they require only one operand. The operands must be a variable name and not a constant.

The increment operator (++) adds one to the current value of the operand and stores the result back into the operand, while the decrement operator (--) subtracts one from the operand and stores the decremented value back into the operand.

There are two different forms of increment and decrement operators. When they are used before the operand, it is termed as prefix, while when used after the operand, they are termed as postfix operators.

Example:

```
int i = 5;
i++;
++i;
--i;
i--;
```

When used in an isolated C statement, both prefix and postfix operators have the same effect, but when they are used in expressions, each of them has a different effect.

In expressions, postfix operator uses the current value and then increments/decrements while in the prefix form the value is incremented/decremented first and then used in the expression. Consider the following examples:

E.g.: `b = a ++;`

this is postfix increment expression. This statement is equivalent to:

```
{b = a;
a = a+1;}
```

E.g. `b = - - a;`

this is prefix decrement expression. This statement is equivalent to:

```
{ a= a-1;
b = a; }
```

Consider the following C code that illustrates the usage of postfix and prefix increment operators.

```
int a = 10; b = 0; // a = 10 and b = 0
a++; // a = 11 and b = 0
b = ++a; // a = 12 and b = 12
b = a++; // a = 13 and b = 12
```

4.9 Conditional Operators

C provides a ternary operator called the conditional operator which is represented by `?:`. The syntax of this operator is given below.

`A?B:C`

Where “A” is a conditional expression resulting in either of the two values – true or false. The value generated by this operator in the expression depends on the value of the conditional expression “A”. If the value of “A” is true then the expression evaluates to “B” otherwise it results in “C”.

4.10 Bitwise Operators

You're aware that a numeric value is stored in binary form in a variable. Data is manipulated at the bit level using bitwise operators. These operators are used to measure bits or to move them left or right. Only integer data types are supported by bitwise operators. The following is a list of the various bit wise operators available in the C language, along with their corresponding meanings.

Operator	Meaning
&	Bitwise Logical AND
	Bitwise Logical OR
^	Bitwise Logical XOR
<<	Left shift
>>	Right shift
~	One's complement

| (Bit-wise OR): Binary operator takes two operands of int type and performs bit-wise OR operation. With assumption that int size is 8-bits:

```
int a = 5; [binary : 0000 0101]
int b = 9; [binary : 0000 1001]
a | b yields [binary : 0000 1101]
```

& (Bit-wise AND): Binary operator takes two operands of int type and performs bit-wise AND operation. With same assumption on int size as above:

```
int a = 5; [binary : 0000 0101]
int b = 9; [binary : 0000 1001]
a & b yields [binary : 0000 0001]
```

^ (Bit-wise Logical XOR): XOR gives 1 if only one of the operand is 1 else 0. With same assumption on int size as above:

```
int a = 5; [binary : 0000 0101]
int b = 9; [binary : 0000 1001]
a ^ b yields [binary : 0000 1100]
```

<< (Shift left): This operator shifts the bits towards left padding the space with 0 by given integer times.

```
int a = 5; [binary : 0000 0101]
a << 3 yields [binary : 0010 1000]
```

>> (Shift right): This operator shifts the bits towards right padding the space with 0.

```
int a = 5; [binary : 0000 0101]
a >> 3 yields [binary : 0000 0000]
```

~ (one's complement operator): It is a unary operator that causes the bits of its operand to be inverted so that 1 becomes 0 and vice-versa. The operator must always precede the operand and must be integer type of all sizes. Assuming that int type is of 1 byte size:

```
int a = 5; [binary: 0000 0101]
~a; [binary: 1111 1010]
```

4.11 Special Operators

The C programming language also includes a variety of special operators that are not found in other languages. The comma operator, size of operator, pointer operators (& and *), and member selection operators (. and ->) are among these operators. When we introduce pointers, we'll talk about pointer operators, and when we talk about structures and unions, we'll talk about member selection operators.

We will discuss comma operator and size of operator in this section.

Comma Operator

This operator is used to link the related expressions together.

Example: `int x, y, z;`

```
z = (x = 10, y=20, x+y);
```

Here, the first statement will create three integer type variables – x, y and z. In the second statement, right-hand side will be evaluated first. Consequently, 10 will be stored in variable x, then 20 will be stored in variable y, and then values in x and y will be multiplied result of which will be stored in variable z. Thus, the value stored in the variable z will be 200 at the end

of execution.

Size of Operator

The size of operator works on variables, constants and even on data types. It returns the number

of bytes the operand occupies in the memory.

Consider the following C code for illustration.

```
sizeof(int); //Gives number of bytes occupied by an
//integer type variable

sizeof(float); //Gives number of bytes occupied by a
//float type variable
```

The output of this code will be 2, 4. Don't get disheartened if you get different result. This is only because the machine on which this program was run allotted 2 bytes for int type and 4 bytes for float type. The result that you get depends on the number of bytes allocated to these types on your machine. Nevertheless, in all cases size of operator returns the number of bytes occupied by its operand on that particular machine.

4.12 Operator Precedence and Associativity

More than one operator can be used in a single sentence. The order in which operators are executed is determined by their precedence. When evaluating expressions involving multiple operators, precedence specifies the order in which operators should be added to the operands.

Depending on the level, operators of the same precedence are evaluated from left to right or right to left. This is referred to as an operator's associativity property. The following is a full list of operator precedence in the C language.

Description	Operators	Associativity
Function expression	()	Left→Right
Array expression	[]	Left→Right
Structure operator	.	Left→Right
Unary Minus	-	Right→Left
Increment/Decrement	++ --	Right→Left
One's complement	~	Right→Left
Negation	!	Right→Left
Address of	&	Right→Left
Value at address	*	Right→Left
Type cast	(type)	Right→Left
Size in bytes	sizeof	Right→Left
Multiplication	*	Left→Right
Division	/	Left→Right
Modulus	%	Left→Right

Addition	+	Left→Right
Subtraction	-	Left→Right
Left shift	<<	Left→Right
Right shift	>>	Left→Right
Less than	<	Left→Right
Less than or equal to	<=	Left→Right
Greater than	>	Left→Right
Greater than or equal to	>=	Left→Right
Equal to	=	Left→Right
Not equal to	!=	Left→Right
Bitwise AND	&	Left→Right
Bitwise XOR	^	Left→Right
Bitwise OR		Left→Right
Logical AND	&&	Left→Right
Logical OR		Left→Right
Conditional	?:	Right→Left
Assignment	=	Right→Left
	* = / = % =	Right→Left
	+ = - = &=	Right→Left
	^ = =	Right→Left
	<< = >> =	Right→Left
Comma	,	Right→Left

Summary

Precedence defines the sequence in which operators are to be applied on the operands, while evaluating the expressions involving more than one operator.

Keywords

Expression: A combination of identifiers and operators according to some rule that yields a value

Operator Preceding: The precedents of an operator determine the order in which expression will be evaluated

Operator: A symbol that works on one or more values to yield another value

The Size of Operator: The size of operator which is used to measure the data sizes. It is a unary compile time operator that is used to return the length of the variable or parenthesized type specifiers.

Self-Assessment

1. Choose the correct statements for unary operator

- A. Unary operators are operators that act upon a single operand to produce a new value.
- B. Unary operator acts like constant
- C. Unary operator is used for logic operations
- D. Above all
- E.

2. Choose the correct statements for operator

- A. constant expressions are evaluated at compile time

- B. An operator perform predefined action on operand.
- C. Main() function is a operator
- D. All of the above

3.Which is correct basic data type

- A. Int
- B. Char
- C. double
- D. Above all

4.In C programming language, which of the following type of operators have the highest precedence

- A. Relational operators
- B. Equality operators
- C. Logical operators
- D. Arithmetic operators

5.Which is not unary operator

- A. ++
- B. --
- C. &
- D. +

6.Relational operator is used with

- A. 1 operand.
- B. 2 operands
- C. Both 1 operand and 2 operand
- D. none of above

7.Which among the following is NOT a logical operator

- A. &&
- B. ||
- C. !
- D. @

8.Find out the wrong statement

- A. a>b
- B. b>=c
- C. a<=> c
- D. a++

9.Which among the following is NOT a relational operator

- A. >=
- B. !=
- C. ==
- D. ||

10. Find out the wrong statement

- A. if (a>b)
- B. if(x=10 && y=20)

- C. if (a<2 || b>30)
 D. if (c=10 ! d>20)
11. Which one is not assignment operator
- A. +=
 B. *=
 C. &
 D. >>=
12. (&=) operator is used for.....
- A. Bitwise exclusive OR and assignment operator
 B. Bitwise inclusive OR and assignment operator.
 C. Bitwise AND assignment operator
 D. None of these
13. Which one is Modulus AND assignment operator
- A. <<=
 B. %=
 C. /=
 D. None of these
14. In c which operators having highest priority
- A. Logical operator
 B. Relational operator
 C. Arithmetic operator
 D. Conditional operator
15. Conditional operator in c is also called
- A. Relational operator
 B. Logical operator
 C. Ternary operator
 D. None of these

Answer for Self Assessment

- | | | | | |
|-------|-------|-------|-------|-------|
| 1. A | 2. B | 3. D | 4. D | 5. D |
| 6. C | 7. D | 8. C | 9. D | 10. C |
| 11. C | 12. C | 13. B | 13. C | 14. C |

Review Questions

1. What are the different classes of operators available in C language?
2. Define the term "Expression". Explain the various types of expression in C.
3. What are the various logical and relational operators supported by C. Explain them with proper examples.
4. Draw a table that will provide a complete list of operators, their precedence level and their rules of association.

5. List down the advantages and limitations of using conditional operator in a C program.
6. Write short notes on:
 - (a) Shorthand assignment operators
 - (b) Bitwise operators
7. Write in detail about the various data type available in C.



Further Readings

B.W. Kernighan and D.M. Ritchie, "The Programming Language", Prentice Hall of India, New Delhi

Byron Gottfried, "Programming With C", Tata McGraw Hill Publishing Company Limited, New Delhi

Greg W Scragg, Genesco Suny, Problem Solving with Computers, Jones and Bartlett, 1997.

R.G. Dromey, Englewood Cliffs, N.J., How to Solve it by Computer, Prentice-Hall International, 1982.

Yashvant Kanetkar, Let us C



Web Links

<https://www.tutorialspoint.com/index.htm>

www.webopedia.com

www.web-source.net

Unit 05: Decision making statements

CONTENTS

Objectives

Introduction

5.1 Designing Structured Programs in C

5.2 Top Down Design

5.3 Type Conversion in C

5.4 Type Modifiers

5.5 Decision-Making Control Statement

5.6 Simple if Statement

5.7 if-else statement

5.8 nested if statement

5.9 Nested else-if statement

5.10 Switch statement

Summary

Keywords

Self Assessment

Answers for Self Assessment

Review Questions

Further Readings

Objectives

- After studying this unit, you will be able to:
- Explain decision making in C
- Explain branching
- Describe if, if-else statement in C
- Explain switch statement
- Understand Type Conversion and Type Modifiers

Introduction

To write a realistic program doesn't mean that collection of the statements arranged in a particular sequence. It requires more than that. Just take an analogy of our real life. Life doesn't go straight all the way as:

1. There are some situations when you have to take decisions like whether to purchase this book or not.
2. There are also some situations where you have to perform the particular action again and again like for better understanding read this unit 5 times continuously.

In the same manner, hardly we write a computer program that may not encounter these situations. Most of the programs require a statement or set of statements to be executed multiple times or not to execute at all, depending on the circumstances.

5.1 Designing Structured Programs in C

A C program is divided into different sections. There are six main sections to a basic c program.

- Documentation
- Link
- Definition
- Global Declarations
- Main functions
- Subprograms

Documentation Section

The documentation section is the part of the program where the programmer gives the details associated with the program. He usually gives the name of the program, the details of the author and other details like the time of coding and description. It gives anyone reading the code the overview of the code.



Example:

**

* File Name: Helloworld.c

* Author: Name

* date: DD/MM/YYYY

* description: a program to display hello world

* no input needed

*/

Link Section

This part of the code is used to declare all the header files that will be used in the program. This leads to the compiler being told to link the header files to the system libraries.



Example: #include<stdio.h>

Definition Section

In this section, we define different constants. The keyword define is used in this part.



Example: #define PI=3.14

Global Declaration Section

The declaration of global variables takes place in this section of the code. This section declares all of the global variables that are used. In this section of the code, the user-defined functions are also declared.



Example: `float area(float r);`

`int a=5;`

Main Function Section

The main role is required in all C programmes. There are two sections of each key role. There are two parts of this: a declaration part and an execution part. All of the variables are declared in the declaration section. The curly brackets are used to start the execution and the curly close bracket is used to end it. The curly braces contain both the declaration and execution sections.

```
int main(void)
{
int a=500;
printf(" %d", a);
return 0;
}
```

Sub Program Section

This section of the software defines all user-defined functions.

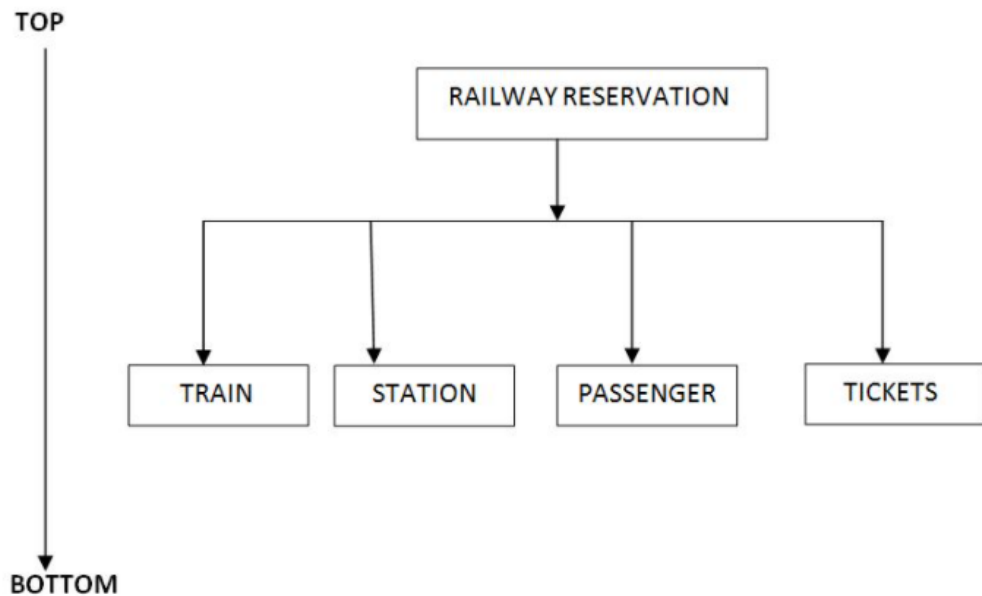
```
int add(int a, int b)
{
return a+b;
}
```

5.2 Top Down Design

A large project is divided into small projects, which are referred to as modules, in this approach. This method of project development is supported by the C programming language. Decomposing a solution into modules in a hierarchical fashion is often a good idea. A top-down approach's basic task is to divide the problem into tasks, which are then divided into smaller sub-tasks, and so on. In this method, the main module is built first, followed by the next level modules. This process is repeated until all of the modules have been developed.

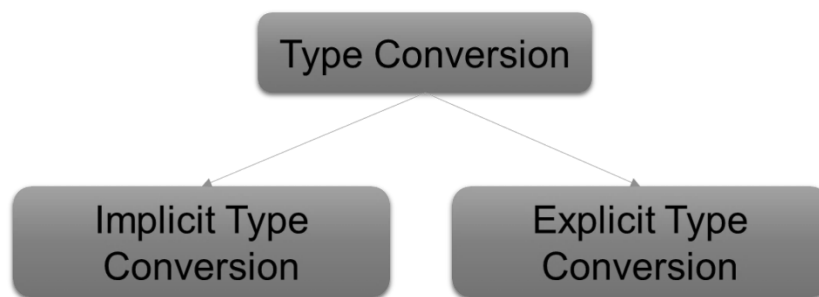
Advantages of top-down approach:

1. In this approach, first, we develop and test most important module.
2. This approach is easy to see the progress of the project by developer or customer.
3. Using this approach, we can utilize computer resources in a proper manner according to the project.
4. Testing and debugging is easier and efficient.
5. In this approach, project implementation is smoother and shorter.
6. This approach is good for detecting and correcting time delays.



5.3 Type Conversion in C

Converting one type of data type to other to perform some operations. Conversion is done on those data types where conversion is possible.



Implicit Type Conversion

This type of conversion is done by compiler whenever necessary automatically without any commands by user. This conversion is performed by compiler when a particular expression contains more than one data type.



Example: `int a = 10;`
`double b = 50.5,sum;`
`sum=a + b;`



Lab Exercise:

```

#include<stdio.h>

int main(){
int a=10;
double b=9.34,sum;

```



```
sum=a+b;
printf("%lf",sum);
return 0;
}
```

Explicit Type Conversion

Without the use of compiler when one data type is changed to another data type, user explicitly defines within the program the data type of the operands in the expression.



Example:

```
double a = 4.5,b=3.5;
//explicitly defined by user
int sum=(int)a+(int)b;
```

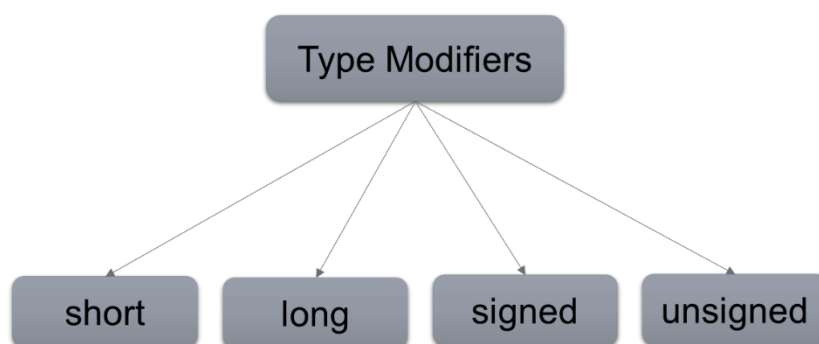


Lab Exercise:

```
#include<stdio.h>
int main(){
double a = 10.5,b=5.5;
//explicitly defined by user
int sum=(int)a+(int)b;
printf("%d",sum);
return 0;
}
```

5.4 Type Modifiers

In c language Data Type Modifiers are keywords used to change the current properties of data type.



Long

This can be used to increased size of the current data.it is applied on int or double data type.



Example: long double a;



Lab Exercise:

```
#include<stdio.h>

int main(){
    long double a ;
    printf("%d",sizeof(a));
    return 0;
}
```

Short

In general int data type occupies different memory spaces for a different operating system; to allocate fixed memory space short keyword can be used.



Example: short int a;



Lab Exercise:

```
#include<stdio.h>

int main(){
    short int a ;
    printf("%d",sizeof(a));
    return 0;
}
```

Unsigned

This keyword can be used to make the accepting values of a data type is positive data type.



```
Example: unsigned int a=10;    //right
        unsigned int a= -50;   //wrong
```



Lab Exercise:

```
#include<stdio.h>

int main(){
    unsigned int a=-10 ;
    printf("%u", a);
    return 0;
}
```

Signed

This keyword accepts both positive and negative value.



Example: unsigned int a=10; //right
 unsigned int a= -50; //right



Lab Exercise:

```
#include<stdio.h>
int main(){
signed int a=-10 ;
printf("%d", a);
return 0;
}
```

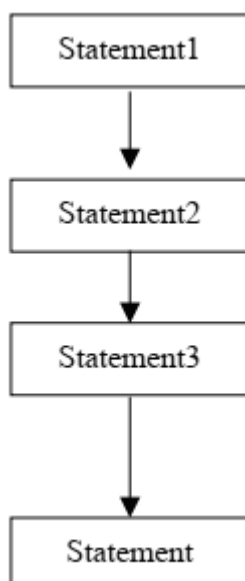
5.5 Decision-Making Control Statement

The statement by which we can control the flow of the program execution is called as control flow statement or program control statement. Program statements may be executed sequentially, selectively or iteratively. The C language provides constructs to support sequence, selection and iteration. The combination of one or more of following constructs explain the flow of the program.

1. Sequence
2. Selection
3. Iteration

Sequence

In the sequence construct, as the name implies, statements are executed sequentially i.e. one after the other. In this, neither the statement are repeated nor in the order of execution changed as shown in Figure.



Programming in C

You probably noticed in the previous unit that a C programme is executed top down, that is, execution begins with the beginning of the main() function and continues, statement by statement, until the end of the main (). In a C programme, the following programme illustrates the sequential execution of statements.

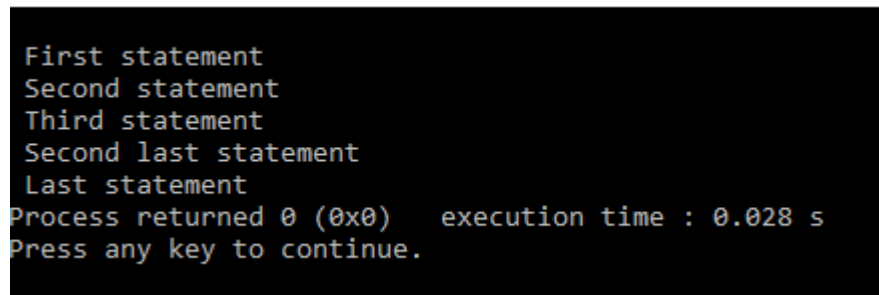


Lab Exercise:

```
#include<stdio.h>

main(){
printf("\n First statement");
printf("\n Second statement");
printf("\n Third statement");
printf("\n Second last statement");
printf("\n Last statement");
}
```

Output : -



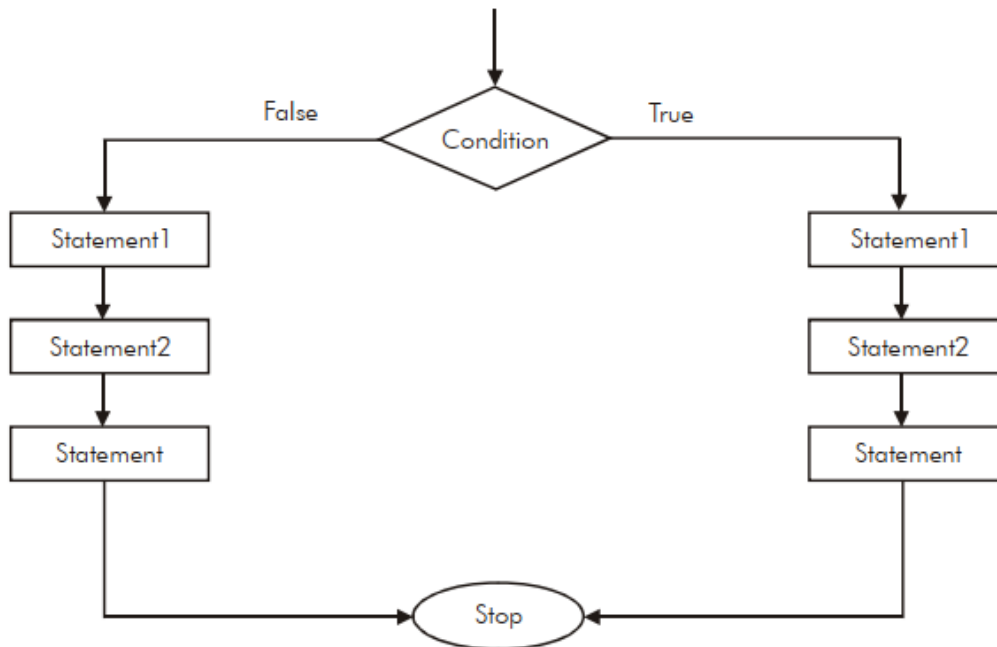
```
First statement
Second statement
Third statement
Second last statement
Last statement
Process returned 0 (0x0) execution time : 0.028 s
Press any key to continue.
```



Notes: Other complex statement may be used in a sequence to demonstrate the concept.

Selection

Depending on the input and the situation involved, only appropriate statements are often executed instead of all of them. The execution of statements in the selection construct is based on a condition test. If the test returns true, you instruct the programme to take one course of action; otherwise, you instruct the programme to take a different course of action. Two or more sets of statements are written in the selection construct, but only one of these sets is executed, as shown in the Figure.



The selection construct can be implemented by means of the if construct. The if construct makes use of relational and logical operators for decision making, as shown in the pseudo code given below:



Lab Exercise:

```

#include<stdio.h>
main(){
int marks=50;
if(marks>40)
    printf("Pass");
else
    printf("Fail");
}
  
```

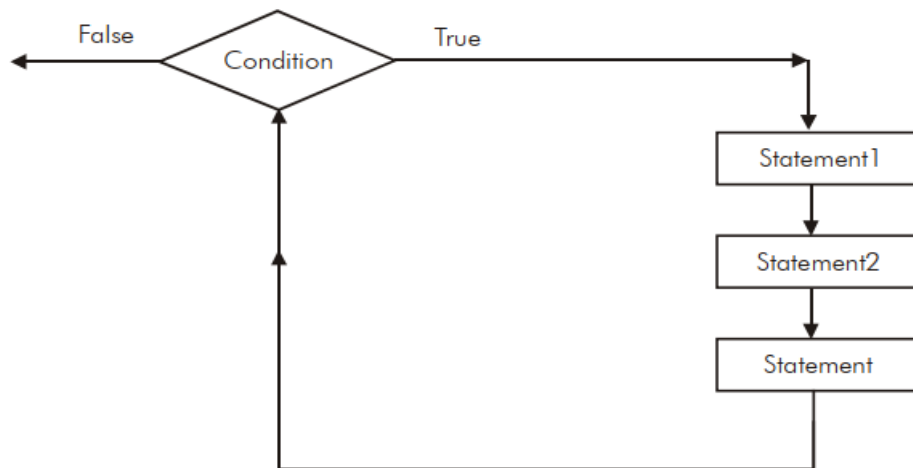
Output: -

```

Pass
Process returned 0 (0x0)   execution time : 0.010 s
Press any key to continue.
  
```

Iteration

The iteration constructs are an efficient method of handling a series of statements that must be repeated a variable number of times. Sometimes the required number of repetitions is known in advance and sometimes the statements repeats over and over until certain specified conditions are met, as shown in the Figure.



The iteration construct is also called as loop. The statements that are to be executed is called as body of the loop and the condition on which a loop terminates is called as exit condition as demonstrated by the program given below:



Lab Exercise:

```
#include<stdio.h>
main(){
int i=0;
while(i<=20){
    printf("\n %d ",i);
    i++;
}
}
```

Output: -

```
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
Process returned 0 (0x0)   execution time : 0.070 s
Press any key to continue.
```

As mentioned earlier, purely sequenced statements are rarely included in a real program. Every programming language must provide statement to support other constructs. C also provides statements to supports these constructs. Let's discuss these constructs in detail.



Notes: What would be the output of following program?

```
main()
{
int x = 20, y = 20 ;
if ( x == y );
printf ( "\n%d %d", x, y );
}
```

5.6 Simple if Statement

In its basic form, the if statement evaluates a test condition (i.e., nothing but an expression) and direct program execution depending on the result of that evaluation. The general form of a simple if statement is as shown below:

```
if (expression)
statement;
```

Where a statement may consist of a single statement, a compound statement or nothing as an empty statement. The expression also referred so as test condition must be enclosed in parentheses, which causes the expression to be evaluated first. If it evaluate to true (i.e., a non-zero value), then the statement associated with it will be executed otherwise ignored and the control will pass to the next statement.



Example:

Consider the following statement:

```
:
if (marks > 9)
printf ("\n Pass");
:
:
```

The above code fragment will printf "Pass" on the monitor if the value of marks is greater than 9. If the value of marks is not greater than 9, the control simply ignore this statement and will pass to the next statement. The following program shows the use of simple if as it accepts the marks of a student and prints his/her result.



Lab Exercise:

```
#include<stdio.h>

void main
{
int marks;
printf( "\n enter your marks");
```

Programming in C

```
scanf("%d", marks);
if (marks>9) /* if construct with test condition */
printf (" \n Pass"); /* statement associated with if */
printf(" Thank you"); //next statement
}
```

output: run 1-> run 2->

Enter your marks : 77 ↵ Enter your marks : 9 ↵

Pass Thank you

Thank you



Notes: "Pass" has been displayed only if the expression evaluated to true otherwise if it evaluated to false, the control ignores the associated statement and executed the next statement i.e., "Thank you".

As mentioned earlier, an if statement can control the execution of multiple statements, called as compound statement or a block. Where a block is a group of two or more statements enclosed in braces. So if these multiple statements are to be executed then they must be placed within a pair of braces, as illustrated by the following program.



Lab Exercise:

Program:

```
#include<stdio.h>

void main()
{
int marks;
printf ("\n Enter your marks: ");
scanf ("%d ", &marks);
if (marks > 39)
{
printf("\n Pass");
printf (" \n Congratulation ...");
}
if (marks <40)
{
printf ("\n Fail");
printf ("\n sorry. Good luck next time ...");
}
printf ("\n Thank you");
}
```

Output: run 1-> run 2->

Enter your marks: 77^d Enter your marks : 20^d

Pass Fail

Congratulations... Sorry. Good luck next time....

Thank you Thank You



Notes: Default an if construct when evaluates to true executes only the first statement associated with it. If multiple statements are not enclosed with in parentheses, results may be unexpected.



Task: Write a program if candidate got more than 50% in year exam screen show "Pass" otherwise "Fail".

5.7 if-else statement

As you must observed in the previous section that the simple if statement executes a single statement or a group of statement when the given expression evaluates to true (i.e. non-zero value). It does nothing when the expression evaluates to false (i.e. a zero value) and simply moves to next statement of the program.

However, if you want a statement or group of statements to be executed. Only when an expression evaluate to false, you can mention this in else section, as shown below the general format of if else statement.

```
if (expression)
```

```
statement1;
```

```
else
```

```
statement 2;
```

If expression evaluates to true, statement 1 is executed. If expression evaluates to false, statement 2 is executed, but never both. Both statement 1 and statement 2, as mentioned earlier, may be single statement, a compound statement, or an empty statement.

Actually, the simple if statement described in previous section is a simplification of its parent statement i.e. if else statement, where the else section in optional. Without it, however, an if-else construct look like a simple if construct.

if-else construct is particularly useful when you have the statements to be executed in both the cases i.e. when the expression evaluates to true or false.



Example: Consider the following statements:

```
:
```

```
if (marks > = 40)
```

```
printf ("\n Pass");
```

```
else
```

```
printf ("\n Fail");
```

```
:
```

Programming in C

The code segment will display “Pass” on the monitor if the value of marks is greater than or equals to 40. If the marks are less than 40 (obviously the else case), then the statement in the else section will be executed and will printf “Fail” on the monitor. Let’s write the Program by using an if-else construct.



Lab exercise:

```
#include<stdio.h>

void main ()
{
    int marks;
    printf (“\n Enter your marks:”);
    scanf (“% d”, & marks);
    if (marks >=40)
    {
        printf (“\n Pass”);
        printf (“\n congratulations....”);
    }
    Else
    {
        printf (“\n Fail”);
        printf (“\n Sorry. Good luck next time ...”);
    }
}
```

Output: run1 → run2 →

Enter your marks: 77→ Enter your marks : 30→

Pass Fail

Congratulations.... Sorry. Good luck next time...



Notes: else section required their own pair of braces as more than one statement is to be executed when the expression evaluates to false.

5.8 nested if statement

The executions of the inner if depends upon its location in the outer construct and upon the value of expression of the outer construct. For instance, consider the following:

```
:
if (expression 1)
{
    if (expression 2)
        statement 1;
    else
        statement 2;
}
```

Unit 05: Decision making statements

In the above segment of code, the inner if executes only if the expression1 evaluates to true. The other possible combination of nested if may take one of the following form:

```
· if (expression1)
{
if (expression2)
statement1;
}
· if (expression1)
{
if (expression 2)
statement1;
else
statement 2;
}
· if(expression1)
{
statement1;
}
else
{
if(exp2)
statement 2;
}
· if (exp 1)
{
statement1;
}
else
{
if (exp2)
statement 2;
else
statement3;
}
if(exp 1)
{
if (exp 2)
statement1;
else
statement 2;
```

```

}
else
{
if (exp3)
statement3;
else
statement4;
}
if (exp1)
{
if (exp2)
statement1;
}
else
{
if (exp3)
statement 2;
}

```

Let's write a couple of programs to explore the various combination of nested if. The following program not only print the request after accepting marks from the students but also print his/her grade.



Lab Exercise:

Program

```

#include<stdio.h>
void main ()
{
int marks;
printf ("\n Enter your marks:");
scanf ("% d",&marks");
if (marks >=40)
{
printf ("\n Pass");
if (marks > = 80)
printf (" with distinction");
}
else
printf ("\n Fail");
}

```

Output: run 1 ->

Enter your marks : 77↵

Pass

Run 2->

Enter your marks : 88↵

Pass with distinction

Run 3->

Enter your marks : 38↵

Fail



Notes: The execution of the inner if only be there if first expression evaluates to true and the message "with distinction" will printf only if both the expressions evaluates to true.

5.9 Nested else-if statement

Imagine a situation where you have to test number of conditions to get the desired results. These types of particular situations requires nestedness of if-else statements up to a deeper level and it may looks like as:

if (expression 1)

statement 1;

else

if (expression 2)

statement 2;

else

if(expression 3)

statement 3;

else

:

statement n;

The following program demonstrates the use of nested if-else statement up to a deeper level. This program will accept the marks of a student and will display the grade accordingly.



Lab Exercise:

Program:

```
# include<stdio.h>
```

```
void main ()
```

```
{
```

```
int marks;
```

```
char grade;
```

```
printf ("\n Enter your marks:");
```

```
scanf ("% d", & marks");
```

```
if (marks > = 90)
```

```
grade = '0';
```

```
else
```

```

if (marks > = 80)
grade = 'D';
else
if( marks > =75)
grade = 'M';
else
if (marks > = 60)
grade = 'I';
else
if (marks > =50)
grade = 'II';
else
if (marks > 40)
grade = 'III';
else
grade = 'F';
printf( "\n Your grade is : % c", grade),
}

```

Output: run 1 →

Enter your marks: 77 ↵

Your grade is: M

Run 2 →

Enter your marks: 39 ↵

Your grade is: F



Notes: This whole section of code is actually one statement that is comprised of six hierarchically nested if else constructs, so there is no need to put them in the braces. At any time during the general top to bottom execution of these expressions, if an expression evaluates to true, then the associated statement will be executed and control flow will pass to the statement immediately following the entire nested chain.

5.10 Switch statement

The switch statement is another convenient tool provided by C to handle the situations in which multiple decisions to be made based on an expression that can have multiple values. The switch is a multiple branch statement that successively tests the value of an expression against a list of case values and when a match is found, the statement associated with the particular case is executed. The general form of a switch-case statement may look like as:

```

switch (expression)
{
case value1: statement1;
case value2: statement2;
case value3: statement3;
:

```

```
case valuen: statementn;
[default: statement x ;]
}
```

statement;

Where switch is a keyword and the expression is any expression that evaluates to an integer value, may be of type int, or char, or long. The case is a keyword followed by value 1, value 2, value n. where value 1, value 2, .. value n may be an integer or character constant, normally referred to as case labels. And the statement1, statement2, .. statementn may be single statement or set of statements, or may be an empty statement.

The switch statement evaluates the expression first and then compare the return value against the values value1, value2,.. valuen, and then one of the following happens:

1. If a case is found whose value matches with the value of the expression then the statement associated with that case is executed.
2. If no match is found then the statement followed by the keyword default is executed.
3. If no match is found and there is no default label as it is an optional case, then no action takes place and control passes to the statement next which is a statement immediately followed the switch statements closing braces.

Consider the following program, which gives you an example of using the switch statement. This program will receive a number between 1 to 5 and will display it's English counterpart.



Lab Exercise:

Program:

```
#include<stdio.h>

void main ()
{
int num;
printf ("\n Enter any number between 1 to 5 : ");
scanf (" % d", &num );
switch (num)
{
case 1 : printf ("\n One");
case 2 : printf ("\n Two");
case 3 : printf ("\n Three");
case 4 : printf ("\n Four");
case 5 : printf ("\n Five");
default : printf ("\n Wrong input");
}
printf ("\n Thank You");
}
```

Output: run 1 → run 2 →

Enter any number between 1 to 5: 2↵ Enter any number between

1 to 5: 4↵

Two Four

Programming in C

Three Five

Four Wrong input

Five Thank you

Wrong input

Thank you

Run 3->

Enter any number between 1 to 5 = 94

Wrong input

Thank you



Notes: There is no need to put braces with the individual case labels as they each contains single statement, although a pair is required to group the entire case section.

Summary

- Most of the programs require a statement or set of statements to be executed multiple times or not to execute at all, depending on the circumstances.
- The statement by which we can control the flow of the program execution is called as control flow statement or program control statement.
- In the sequence construct, as the name implies, statements are executed sequentially i.e. one after the other.
- In selection construct, the execution of statements depends upon a condition test.
- The iteration constructs are an efficient method of handling a series of statements that must be repeated a variable number of times.
- If multiple statements are to be executed then they must be placed within a pair of braces.
- A simple if or if else construct may be placed within another if or if-else construct.
- The switch statement is another convenient tool provided by C to handle the situations in which multiple decisions to be made based on an expression that can have multiple values.

Keywords

Conditional statement: A statement that evaluates to either true or false.

Continue statement: The statement that ignores execution of further statements and forces the loop to evaluate the loop condition once again.

Default statement: An optional statement in a switch that is executed if none of the conditions evaluates to true.

Switch statement: A multi-selection statement that branches to that statement whose specified condition evaluates to true.

Self Assessment

1. Every C program must have
 - A. Data type
 - B. Constant
 - C. Main Function
 - D. None of the above

2. Choose the right statement
- A. User defined functions is known as sub-program.
 - B. Sub program is a constant in C program
 - C. Sub program and main function is same
 - D. All of above
3. Global declaration is _____
- A. A sub program
 - B. A data type
 - C. A part of C program where we can declare global variables.
 - D. All of the above
4. How many type of conversion are there in c?
- A. 4
 - B. 2
 - C. 3
 - D. 1
5. Which conversion also called Automatic Type Conversion?
- A. Implicit Type Conversion
 - B. Explicit Type Conversion
 - C. All of above
 - D. None of the above
6. What will be output for the following code?
- ```
#include<stdio.h>

int main()
{
 double x = 2.2;
 int sum = (int)x + 5;
 printf("sum = %d", sum);
 return 0;
}
```
- A. 2
  - B. 2.2
  - C. 5
  - D. 7
7. Which type of conversion is NOT accepted?
- A. From char to int
  - B. From float to char pointer

- C. From negative int to char
- D. From double to char

8. Type modifiers in C are

- A. Int, main
- B. Sub program
- C. Short, long, signed and unsigned
- D. All of above

9. What will be output for the following code?

```
#include <stdio.h>

int x;

void main()
{
 if (x)
 printf("hi");
 else
 printf("how are u");
}
```

- A. hi
- B. how are u
- C. compile time error
- D. error

10. What will be output for the following code?

```
#include <stdio.h>

void main()
{
 int x = 5;
 if (true);
 printf("hello");
}
```

- A. It will display hello
- B. It will throw an error
- C. Nothing will be displayed
- D. Compiler dependent

11. What will be the output of following program?

```
#include <stdio.h>

void main()
{
```

```
char val=1;
if(val--==0)
 printf("TRUE");
else
 printf("FALSE");
}
```

- A. FALSE
- B. Error
- C. TRUE
- D. None

12. Which of the following is an invalid if-else statement?

- A. if (if (a == 1)){}
- B. if (func1 (a)){}
- C. if (a){}
- D. if ((char) a){}

13. The label in goto statement is same like

- A. Case in switch statement
- B. Initialization in for loop
- C. Continuation condition in for loop
- D. All of them

14. What will be the output of following program?

```
#include <stdio.h>
void main()
{
 int a=10;
 switch(a){
 case 5+5:
 printf("Hello ");
 default:
 printf("World\n");
 }
}
```

- A. Hello
- B. World
- C. Hello World
- D. Error

15. What will be the output of following program?

```
#include <stdio.h>
```

```
void main()
{
 short day=2;
 switch(day)
 {
 case 2: | | case 22:
 printf("%d nd",day);
 break;
 default:
 printf("%d th",day);
 break;
 }
}
```

- A. 2 nd
- B. 22 nd
- C. Error
- D. 2 nd 22 nd

16. What will be the output of following program?

```
#include <stdio.h>

int main()
{
 int x = 2;
 switch (x)
 {
 case 1: printf("x =1");
 break;
 case 2: printf("x= 2");
 break;
 case 3: printf("x= 3");
 break;
 default: printf("x other than 1, 2 and 3");
 break;
 }
 return 0;
}
```

- A. x=1
- B. x=2
- C. x=3
- D. All of above

### **Answers for Self Assessment**

1. C            2. A            3. C            4. B            5. A
6. D            7. B            8. C            9. B            10. B
11. A           12. A           13. A           14. C           15. C
16. B

### **Review Questions**

1. Write a program using if-else statement.
2. Explain nested-if statement with example.
3. What do you mean by switch statement? How it used
4. A five-digit number is entered through the keyboard. Write a program to obtain the reversed number and to determine whether the original and reversed numbers are equal or not.
5. Write a program to check whether a triangle is valid or not, when the three angles of the triangle are entered through the keyboard. A triangle is valid if the sum of all the three angles is equal to 180 degrees.
6. Given the length and breadth of a rectangle, write a program to find whether the area of the rectangle is greater than its perimeter. For example, the area of the rectangle with length = 5 and breadth = 4 is greater than its perimeter.
7. What is the use of if-else statement?
8. Define selection in c programming.
9. Write a program in C to enter five integer values as age of five boys and calculate the average age of all the boys.
10. Write a program to calculate the area of a square. All values enter with the help of keyboard.



### **Further Readings**

- Ashok N. Kamthane, "Programming with ANCI & Turbo C", Pearson Education, Year of Publication, 2008.
- B.W. Kernighan and D.M. Ritchie, "The Programming Language", Prentice Hall of India, New Delhi.
- Byron Gottfried, "Programming with C", Tata McGraw Hill Publishing Company Limited, New Delhi.
- Greg W Scragg, Genesco Suny, Problem Solving with Computers, Jones and Bartlett, 1997.
- R.G. Dromey, Englewood Cliffs, N.J., How to Solve it by Computer, Prentice-Hall International, 1982.
- Yashvant Kanetkar, Let us C



### **Web Links**

<https://www.tutorialspoint.com/index.htm>

[www.webopedia.com](http://www.webopedia.com)

[www.web-source.net](http://www.web-source.net)